



Feito em PHP nativo

Controle Financeiro



2026.01

Brendo Henrique
Claudio Vasconcellos
Otávio Augusto
Samir Batista
Tiago Veras

17/06/2006



Evolução da Arquitetura de Dados (APGs)

APG-1: Teve papel crucial na nossa introdução à Engenharia de Requisitos. Aprendemos a definir o problema de software e a documentar requisitos de forma estruturada, dando a base teórica de planejamento que usamos no projeto final.

APG-2: Foi o nosso primeiro laboratório prático em equipe. Nela, aplicamos o padrão arquitetural MVC de forma nativa no PHP e fomos forçados a usar o GitFlow de forma rígida para gerar nossa primeira Release estável.

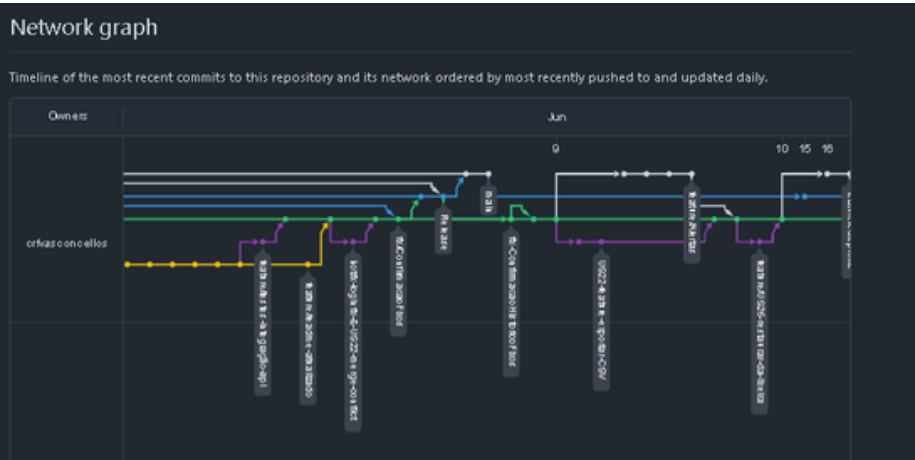


Metodologias Ágeis Aplicadas

Foram aplicados princípios de metodologias ágeis, organizando o trabalho por meio de Sprints, definição de User Stories e priorização de funcionalidades no Product Backlog.

Essa abordagem permitiu entregas incrementais, melhor comunicação entre a equipe e maior adaptação às mudanças ao longo do projeto, proporcionando uma experiência prática próxima à realidade do desenvolvimento de software.

Backlog do Produto				
As estimativas seguem a Sequência de Fibonacci.				
ID	User Story	Pts	Status	Crterios de Aceitaão
US01	Adicionar despesa para controlar gastos	5	✓	Validar nome, valor positivo e data
US02	Visualizar despesas para acompanhar gastos	5	✓	Lista atualizada com nome, valor e data
US03	Excluir despesa para corrigir erros	3	✓	Remoção imediata da lista
US04	Ver total de gastos	3	✓	Somatório automático
US05	Editar despesa	5	✓	Alteração de nome, valor e data
US06	Validar campos obrigatórios	3	✓	Impedir dados inválidos
US07	Persistência em banco SQL	8	✓	CRUD completo
US08	Listar histórico de despesas	5	✓	Histórico funcional
US09	Categorizar despesas	5	✓	Associação de categorias
US10	Atualizar interface	5	✓	Interface responsiva
US11	Adicionar saldo	5	✓	Cadastro de entradas
US12	Indexar comprovante PDF	3	✓	Upload funcional
US13	Filtrar despesas	3	✓	Filtro por categoria
US14	Buscar despesas por nome	2	✓	Busca textual
US15	Exibir gastos por categoria	5	✓	Totais agrupados
US16	Destacar maior despesa	3	✓	Identificação automática
US17	Despesas recorrentes mensais	5	✓	Geração automática
US18	Saldos recorrentes mensais	3	✓	Entradas automáticas
US19	Remover gastos fixos do histórico	3	✓	Remover registros vinculados
US20	Implementar modo escuro	5	✓	Alternância de tema
US21	Exibir média de gastos mensais	3	✓	Média automática
US22	Exportar e importar transações via CSV	5	✓	Download CSV com filtros + upload com criação em lote
US23	Editar perfil do usuário (nome e senha)	5	✓	Modal de perfil com validação de senha
US24	Alertas visuais de orçamento ao se aproximar do limite	3	✓	Notificação no dashboard a 75%, 95% e acima
US25	Comparativo de gastos entre mês atual e anterior	5	■	Card com variação percentual e indicador visual
US26	Restaurar transação excluída da lista	3	✓	Botão restaurar no histórico, restaurar excluídas...
Concluído • Planejado				



Foram utilizadas principalmente princípios do Scrum:

- Product Backlog;
- User Stories;
- Sprints;
- Priorização de funcionalidades;
- Entregas incrementais.



Lições e Desafios Vivenciados

Um dos principais **desafios** enfrentados foi a **sobrescrita de código** e **Merge** durante a integração das funcionalidades desenvolvidas pelos integrantes, além das dificuldades iniciais com a **utilização do GitFlow**.

Para solucionar esses problemas, passamos a **organizar melhor o uso das branches**, **revisar as integrações com mais atenção** e **manter uma comunicação** mais frequente entre a equipe.

A **organização do projeto** através das **Sprints** e **User Stories**, aliada às **revisões de código** antes das integrações, contribuiu significativamente para **reduzir erros** e **melhorar a colaboração** entre os integrantes.

Como resultado, adquirimos maior experiência com controle de versão e desenvolvimento colaborativo, reduzindo erros e tornando o processo de desenvolvimento mais eficiente.



Pontos Positivos da Disciplina

Aspectos positivos

- Experiência prática próxima à realidade da indústria de software;
- Desenvolvimento de habilidades de trabalho em equipe;
- Aplicação de metodologias ágeis ;
- Aprendizado do uso do GitHub e GitFlow em projetos colaborativos;
- Integração entre teoria e prática por meio da construção de um produto real.

A disciplina proporcionou uma experiência muito próxima do ambiente profissional, permitindo aplicar conceitos de metodologias ágeis, controle de versão e desenvolvimento em equipe. Além disso, possibilitou o desenvolvimento de competências técnicas e interpessoais importantes para a atuação no mercado de software.



Pontos Negativos da Disciplina

Aspectos negativos

- Necessidade de aprender PHP, Git e GitHub de forma autônoma;
- Curva de aprendizado inicial elevada;
- Pouca flexibilidade na escolha de linguagens e tecnologias;
- Tempo limitado para adaptação às ferramentas utilizadas.

Apesar de a disciplina ter como foco os processos e práticas de Engenharia de Software, e não o ensino de programação em si, a **necessidade de adquirir conhecimentos técnicos** sobre PHP, Git e GitHub de forma autônoma foi um dos **principais desafios** enfrentados pela equipe.

Além disso, a **limitação na escolha de linguagens e tecnologias** reduziu a possibilidade de explorar ferramentas com as quais os integrantes possuíam maior familiaridade ou interesse.



Nota da experiência da Disciplina

Nota atribuída: **8,5/10**

Justificativa:

- Experiência prática próxima à realidade da indústria de software;
- Desenvolvimento de habilidades de trabalho em equipe e metodologias ágeis;
- Aprendizado sobre GitFlow e desenvolvimento colaborativo;
- Necessidade de aprendizado autônomo de algumas tecnologias;
- Pouca flexibilidade na escolha das linguagens e ferramentas utilizadas.

Como equipe, atribuímos a nota 8,5 para a disciplina. Consideramos que ela **proporcionou uma experiência** muito **valiosa** e **próxima** do mercado de trabalho, permitindo aplicar conceitos importantes de Engenharia de Software.

Entretanto, aspectos como **a necessidade de aprendizado autônomo** de determinadas tecnologias e a **limitação na escolha das ferramentas** utilizadas influenciaram nossa avaliação.



Sugestões de Melhoria para a Disciplina

- Oferecer maior liberdade na escolha das linguagens e tecnologias utilizadas nos projetos;
- Incentivar a diversidade de soluções tecnológicas, permitindo que os alunos explorem ferramentas alinhadas aos seus interesses e conhecimentos prévios.

Nossa principal sugestão é oferecer **maior flexibilidade na escolha das tecnologias** utilizadas nos projetos. Acreditamos que isso permitiria aos alunos **explorar diferentes ferramentas, aproveitar conhecimentos prévios e desenvolver soluções mais alinhadas às demandas e tendências** do mercado de software.



Controle Financeiro

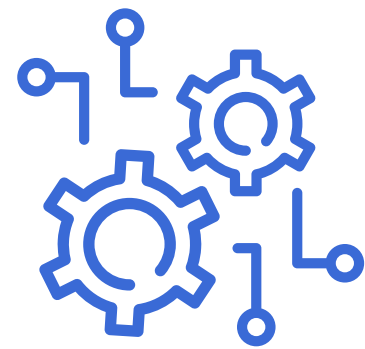
Aplicação web para registro, acompanhamento e análise de gastos pessoais.

Objetivo

Auxiliar usuários no **controle de receitas** e **despesas**, promovendo uma melhor organização financeira.

Problemas Resolvidos

- Dificuldade em controlar gastos pessoais;
- Falta de organização financeira;
- Necessidade de acompanhar despesas recorrentes;
- Pouca visibilidade sobre hábitos de consumo.



Funcionalidades Principais



Cadastro de despesas



Registro de
despesas e entradas



Upload de
comprovantes em
PDF



Despesas recorrentes



Dashboard financeiro

Cadastro despesas

- Nome;
- Descrição;
- Valor;
- Data;
- Categoria;
- Documento.

A aba de despesa permite criar uma despesa guardando o título, o valor da despesa, a data que ocorreu, a categoria para facilitar na filtragem e categorização, e um documento PDF para guardar comprovantes por exemplo.

Nova Despesa

Data Inicial



Data Final (Recorrência)



Símbolo (Ícone)

 Documento (Padrão)

▼

Comprovante (PDF opcional)

Escolher arquivo

Nenhum arquivo escolhido

Registrar Despesa

Registro de saldos e entradas

- Nome;
- Descrição;
- Valor;
- Data;
- Categoria;
- Documento.

A aba de saldo permite **guardar** o **título**, o **valor da recebido**, a **data** do **recebimento**, a **categoria** para facilitar na **filtragem** e **categorização**, e um **documento** PDF para **guardar comprovantes** por exemplo.

Novo Saldo

Data Inicial



Data Final (Recorrência)



Símbolo (Ícone)

 Dinheiro (Padrão)

▼

Comprovante (PDF opcional)

Escolher arquivo

Nenhum arquivo escolhido

Adicionar Saldo

Despesas recorrentes

As **despesas recorrentes automatizam** a criação de despesas recorrentes, criando as despesas todo mes no dia determinado e permitindo uma **fácil visualização** de todas as despesas recorrentes.

Novo Recorrente Fixo

Despesa Fixa

Saldo Fixo

Título / Nome

Descrição (opcional)

Valor Mensal (R\$)

Dia do Mês

Dia 17

Primeiro lançamento

17/06/2026

Símbolo (ícone)

Recorrente (Padrão)

Criar Recorrente Fixo

Recorrentes Fixos

Despesas e saldos que se repetem todo mês automaticamente.

Novo Recorrente

Aluguel

DESPESA

- R\$

1.800,00

Moradia

Dia 10 de cada mês —

Desde 03/2026

Streaming

DESPESA

- R\$

55,90

Netflix + Spotify

Dia 15 de cada mês — Desde

03/2026

Internet Fibra

DESPESA

- R\$

149,90

Plano 500MB

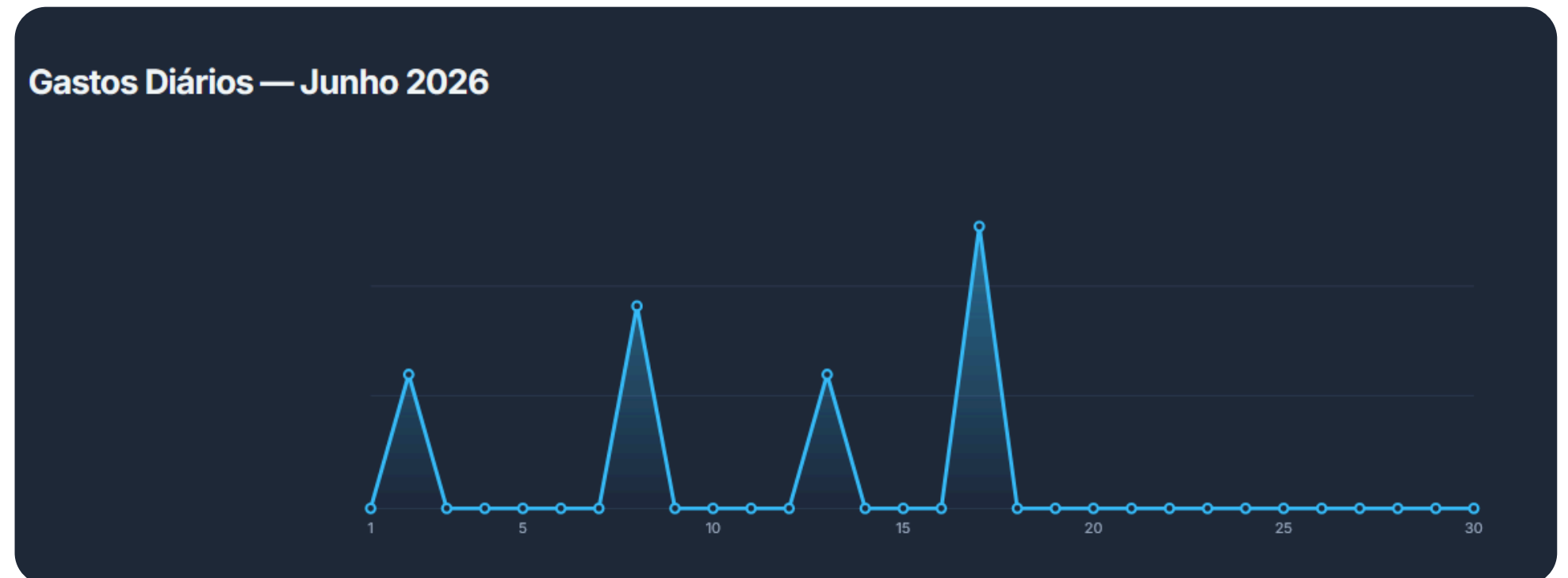
Dia 20 de cada mês — Desde

03/2026

13

Dashboard Financeiro

- Resumo financeiro;
- Gastos mensais;
- Maior despesa do período;
- Gastos por categoria;
- Media de gastos mensais.



O dashboard **apresenta informações** importantes para **auxiliar** na tomada de decisões financeiras.



Prova de conhecimento



Prova de conhecimento

Caso iniciássemos um novo projeto para um cliente com um problema desconhecido.

Nesta etapa, apresentaremos as **estratégias**, **práticas** e **decisões** que utilizaríamos para **compreender as necessidades do cliente**, **organizar a equipe**, **planejar o desenvolvimento** e **garantir a qualidade da solução proposta**, utilizando princípios de Engenharia de Software e metodologias ágeis.

Estratégias para entender o problema e os requisitos

- Realizar reuniões de levantamento de requisitos com o cliente;
- Identificar os objetivos e necessidades do negócio;
- Utilizar entrevistas e questionários para esclarecer dúvidas;
- Elaborar User Stories para representar as funcionalidades desejadas;
- Definir e validar requisitos funcionais e não funcionais;
- Revisar periodicamente os requisitos junto ao cliente para garantir alinhamento.

Inicialmente, realizamos reuniões com o cliente para **entender o problema** e **os objetivos do sistema**. Em seguida, utilizamos entrevistas e questionários para levantar os requisitos.

Depois, organizamos essas necessidades em **User Stories**, definindo também **requisitos funcionais e não funcionais**. Criamos **protótipos simples** para validação e mantemos **revisões contínuas** com o cliente para garantir alinhamento durante todo o projeto.

Requisitos prioritários do produto

- Levantar todos os requisitos e organizar em um backlog inicial
- Classificar os requisitos por valor de negócio e impacto para o usuário
- Identificar funcionalidades essenciais (MVP) para a primeira entrega
- Considerar dependências entre requisitos
- Validar prioridades com o cliente
- Revisar prioridades a cada sprint conforme o feedback.

Para definir os requisitos prioritários, inicialmente **levantamos todas as funcionalidades** e **organizamos em um backlog**. Em seguida, **classificamos** esses requisitos **com base no valor para o usuário e no impacto no negócio**, além do esforço necessário para implementação.

Separamos **o que é essencial** do que **pode ser implementado depois**. Também identificamos o **MVP**, ou seja, as funcionalidades mínimas para uma primeira entrega funcional.

Por fim, **validamos essas prioridades** com o cliente e **ajustamos continuamente** ao longo das sprints conforme o feedback do projeto.

Validação dos requisitos

- Revisar os requisitos junto ao cliente
- Utilizar protótipos (wireframes) para validação visual do sistema
- Criar User Stories com critérios de aceitação claros
- Realizar reuniões de revisão (review) a cada sprint
- Simular cenários de uso (testes de validação).
- Ajustar requisitos conforme feedback recebido.

Para validar os requisitos, **realizamos revisões constantes** com o cliente, garantindo que o entendimento esteja correto.

Também **utilizamos protótipos** para facilitar **a validação visual** do sistema e **criamos User Stories** com critérios de aceitação bem definidos.

Além disso, fazemos **revisões ao final de cada sprint** e **simulamos cenários de uso** para verificar se o sistema atende às necessidades.

Com base no feedback, **ajustamos os requisitos** sempre que necessário, garantindo alinhamento contínuo com o cliente.

Definição e validação das tecnologias

- Levantar requisitos funcionais e não funcionais do sistema;
- Analisar requisitos técnicos (desempenho, segurança, escalabilidade);
- Pesquisar tecnologias adequadas ao problema (stack);
- Avaliar experiência da equipe com as tecnologias;
- Comparar alternativas com base em custo, facilidade e manutenção;

Analisamos os requisitos funcionais e não funcionais do sistema, como desempenho, segurança e escalabilidade.

Em seguida, pesquisamos e comparamos tecnologias que melhor atendem a esses requisitos, considerando também a experiência da equipe com cada uma delas.

Por fim, validamos a escolha em conjunto com a equipe e ajustamos caso surjam limitações durante o desenvolvimento do projeto.

Importância do planejamento das iterações (Sprints)

- Permite dividir o projeto em entregas menores e mais gerenciáveis;
- Facilita o acompanhamento do progresso do sistema;
- Garante priorização das funcionalidades mais importantes;
- Reduz riscos ao permitir ajustes a cada ciclo;
- Melhora a comunicação entre equipe e cliente;
- Possibilita feedback contínuo e melhorias constantes;
- Aumenta previsibilidade de prazo e entregas.

Sim, o planejamento das iterações, como as Sprints no Scrum, é essencial para o projeto. Ele permite dividir o desenvolvimento em partes menores, facilitando o controle e a organização das entregas.

Além disso, ajuda a priorizar as funcionalidades mais importantes e reduz riscos, já que o sistema pode ser ajustado a cada ciclo com base no feedback.

Também melhora a comunicação com o cliente/equipe e aumenta a previsibilidade das entregas, tornando o desenvolvimento mais eficiente e alinhado às necessidades do projeto.

Importância do valor em uma iteração

- Foca nas funcionalidades mais importantes para o usuário;
- Prioriza o que gera maior impacto no sistema;
- Evita esforço em funcionalidades de baixo valor;
- Facilita a entrega de um MVP mais rápido;
- Aumenta a satisfação do cliente.

A definição de valor em uma iteração **é importante** porque **garante foco no que realmente importa** para o usuário.

Ajudando **a priorizar funcionalidades de maior impacto**, **evitando desperdício de esforço** e **facilitando a entrega de um MVP**.

No final, isso aumenta a satisfação do cliente com entregas mais relevantes.

Importância de seguir padrões (ex: MVC)

- Organiza melhor o código e o projeto;
- Separa responsabilidades (Model, View e Controller);
- Facilita manutenção e evolução do sistema;
- Melhora o trabalho em equipe;
- Reduz acoplamento entre partes do sistema;
- Torna o código mais reutilizável e testável.

Seguir padrões como o MVC é importante porque organiza melhor o sistema e separa responsabilidades.

Isso facilita a manutenção, o trabalho em equipe e a evolução do projeto.

Além disso, reduz o acoplamento entre as partes e torna o código mais fácil de testar e reutilizar.

Importância de arquitetura baseada em serviços (API REST)

- Permite comunicação entre sistemas de forma padronizada;
- Facilita integração com outras aplicações;
- Torna o sistema mais escalável e flexível;
- Separa frontend e backend;
- Melhora reutilização das funcionalidades via endpoints;
- Facilita manutenção e evolução do sistema.

A arquitetura baseada em serviços com API REST é importante porque padroniza a comunicação entre sistemas.

Ela permite integrar diferentes aplicações, separa frontend e backend e torna o sistema mais escalável e flexível.

Além disso, facilita a manutenção e a reutilização das funcionalidades através de endpoints.

Importância dos Testes Unitários

- Garantem que cada parte do sistema funcione corretamente;
- Facilitam a identificação de erros mais cedo;
- Evitam regressões ao alterar o código;
- Melhoram a qualidade e confiabilidade do sistema;
- Auxiliam na manutenção e evolução do software;
- Permitem validar regras de negócio de forma isolada.

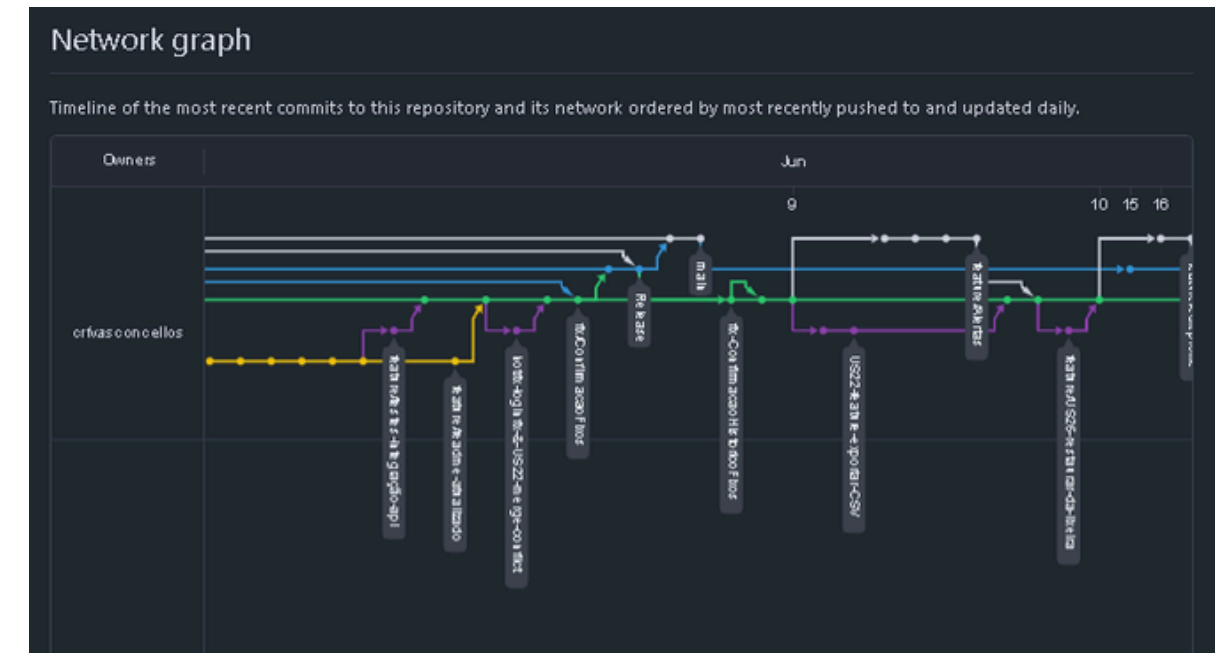
Os testes unitários são importantes porque **garantem que cada parte do sistema funcione corretamente** de **forma isolada**.

Eles **ajudam a identificar erros** mais cedo, **evitam falhas** quando o código é alterado e **aumentam a confiabilidade** do sistema.

Além disso, **facilitam a manutenção** e asseguram que as regras de negócio estejam sendo aplicadas corretamente.

Importância do Controle de Versão

- Evita perda de código e alterações importantes;
- Permite trabalho em equipe de forma organizada;
- Registra o histórico de mudanças do sistema;
- Facilita voltar versões anteriores em caso de erro;
- Ajuda no gerenciamento de novas funcionalidades;
- Reduz conflitos durante o desenvolvimento.



O controle de versão é importante porque **evita perda de código**, **registra o histórico de alterações** e **facilita o trabalho em equipe**.

Ele é **realizado com o Git**, utilizando uma estrutura como o **GitFlow**, onde **cada funcionalidade é desenvolvida em uma branch separada**.

As mudanças são registradas por meio de **commits** e **depois integradas ao sistema principal** através de **merges**, garantindo organização e segurança no desenvolvimento.

Fluxo de trabalho com GitFlow

- Organiza o desenvolvimento em branches bem definidas;
- *main* contém a versão estável do sistema;
- *develop* reúne funcionalidades em desenvolvimento;
- *feature/** é usada para novas funcionalidades;
- Facilita trabalho em equipe sem conflitos;
- Permite controle e revisão antes de ir para produção;
- Ajuda a manter o histórico do projeto limpo e organizado.

O GitFlow organiza o fluxo de trabalho separando o código em branches específicas, como *main*, *develop* e *feature*.

Isso permite que cada funcionalidade seja desenvolvida de forma isolada, reduzindo conflitos e facilitando o trabalho em equipe.

Além disso, garante que apenas versões estáveis cheguem à produção, mantendo o projeto mais organizado e controlado.

Release do produto

- É a entrega de uma versão do sistema pronta para uso
- Contém um conjunto de funcionalidades testadas e validadas
- Representa um “marco” no desenvolvimento do produto
- Pode ser uma versão inicial (MVP) ou versões evolutivas
- É publicada para uso dos usuários finais.

Release do produto é a entrega de uma versão do sistema que já está pronta para uso.

Ela reúne funcionalidades testadas e validadas, representando um marco no desenvolvimento.

Cada release pode trazer novas melhorias ou versões iniciais do sistema, sempre passando por testes antes de ser disponibilizada ao usuário final.



Obrigado!

